



Vietnam National University of HCMC
International University
School of Computer Science and Engineering



Object Oriented Programming

Inheritance

Instructor: Le Thi Ngoc Hanh, Ph.D

ltnhanh@hcmiu.edu.vn

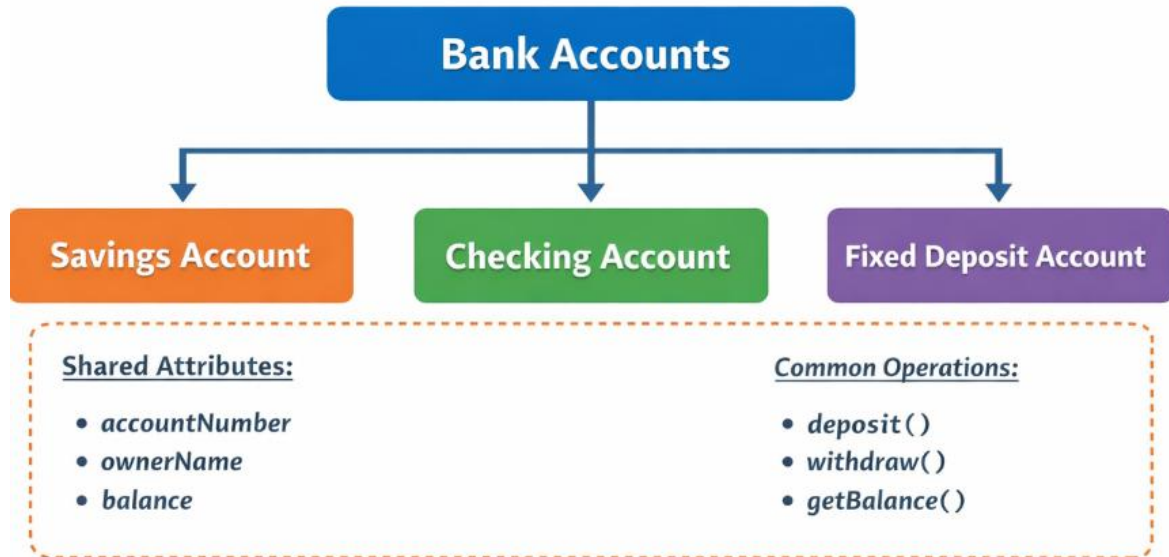
Semester – Spring 2026

Lecture Objectives

- ✧ Concept of inheritance and its purpose in OOP
- ✧ Superclass and subclass relationships
- ✧ Controlled access in Inheritance
- ✧ Constructors in subclasses

Example

In many software systems, different objects share **common attributes and behaviors**.



Problem without Inheritance

★ **Code duplication**

Multiple classes contain identical implementations.

★ **Maintenance difficulties**

A change in deposit logic must be updated in several classes.

★ **Higher probability of errors**

Different classes may accidentally implement the same method differently.

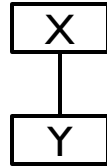
➡ Object-oriented design introduces **inheritance** to solve this problem.

Definition of Inheritance

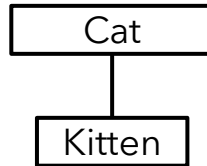
- ☼ **Inheritance** is a mechanism in object-oriented programming where a new class derives properties and behaviors from an existing class.
 - A general class defines common attributes and behaviors.
 - More specialized classes extend this general class and inherit its features.
- ☼ Advantages: promotes code reuse; reduces redundancy; Improves maintainability and extensibility of software systems
- ☼ Inheritance establishes logical **hierarchical relationships between classes**

Inheritance

- Objects in the real world and in mathematics can be further classified by hierarchical taxonomies.
- For the taxonomy



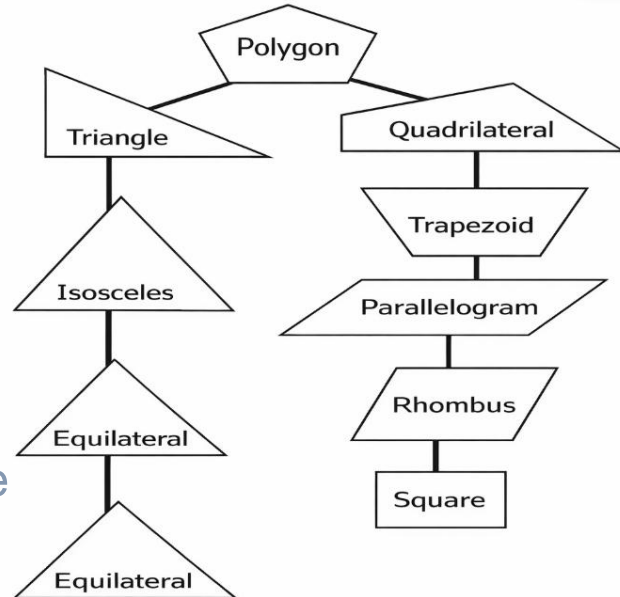
we say "every Y is-a X". For example,



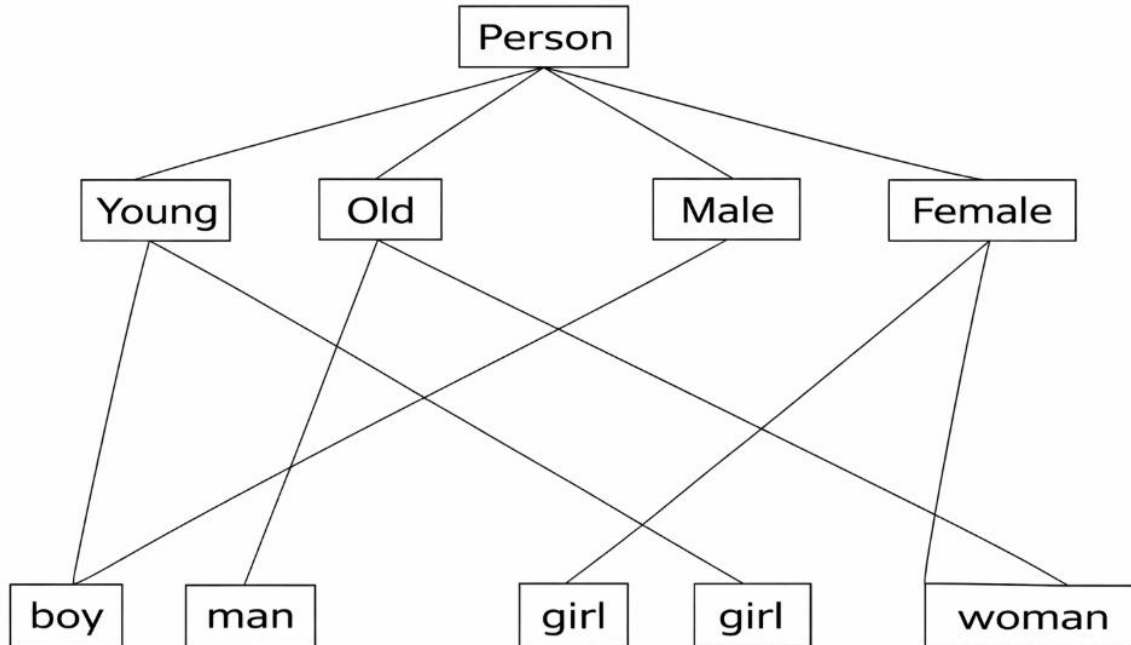
"every kitten is a cat"

Taxonomy

- ★ A **taxonomy** is a hierarchical classification where:
- Each subclass belongs to **exactly one parent category**
 - The relationships represent **IS-A relationships**
 - The hierarchy forms a **tree structure**
 - A child node should have **only one parent**



Taxonomy



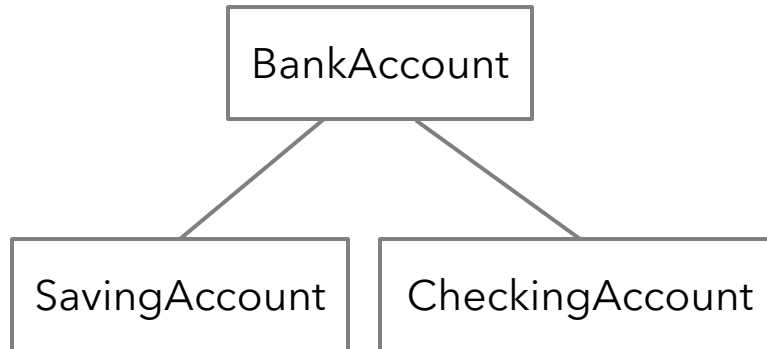
Superclasses and Subclasses

In an inheritance hierarchy, two important terms are used:

- ★ **Superclass:** A superclass (also called parent class or base class) is a class that contains **common attributes and methods** shared by other classes.
- ★ **Subclass:** A subclass (also called child class or derived class) is a class that **inherits attributes and methods from a superclass**.
- ★ The subclass can: **use inherited methods; define additional methods; modify inherited behavior**

Class Hierarchy Example

- ☾ Inheritance **should always** represent logical specialization relationships.



- ➔ This relationship is known as an IS-A relationship.

Java Syntax for Inheritance

In Java, inheritance is implemented using the keyword “**extends**”.

Syntax:

```
class SubclassName extends SuperclassName {  
  
}
```

Example:

```
public class SavingsAccount extends BankAccount {  
  
}
```

- ✧ The class SavingsAccount inherits all accessible members of the class BankAccount.
- ✧ This allows the subclass to reuse code defined in the superclass.

Example of a Superclass

```
public class BankAccount {  
  
    protected double balance;  
  
    public void deposit(double amount) {  
        balance += amount;  
    }  
  
    public double getBalance() {  
        return balance;  
    }  
}
```

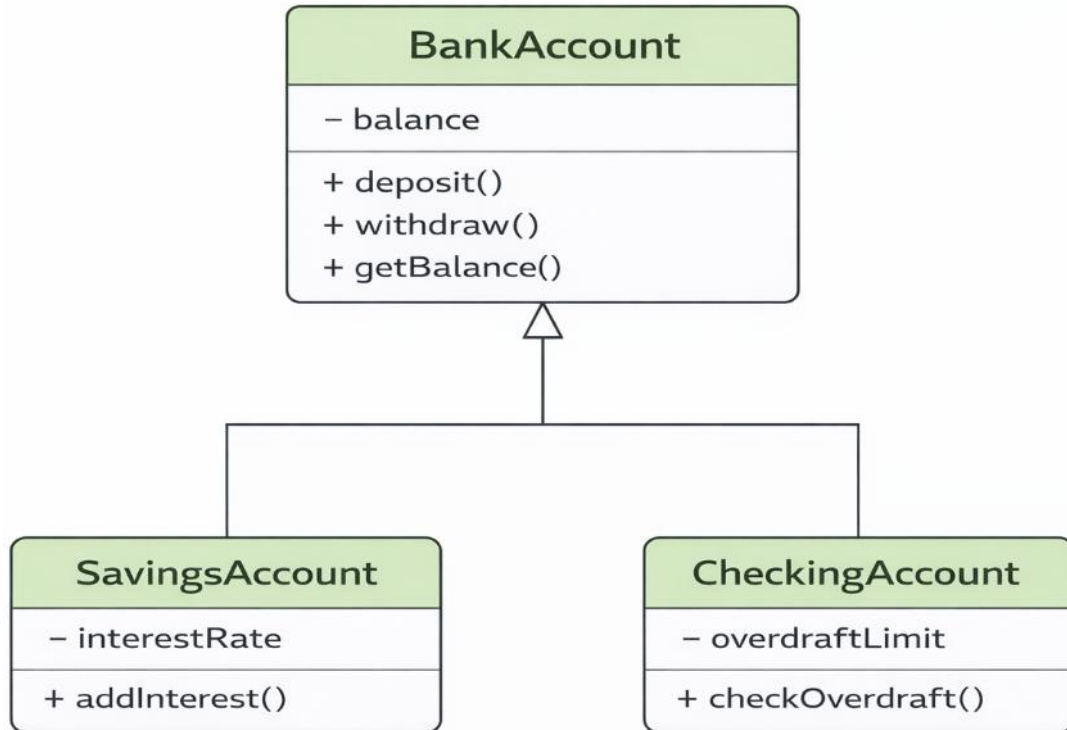
Example of a Subclass

```
public class SavingsAccount extends BankAccount {  
  
    private double interestRate;  
  
    public void addInterest() {  
        balance += balance * interestRate;  
    }  
}
```

What Does a Subclass Inherit?

- ☾ When a subclass extends a superclass, it inherits the **accessible members** of the superclass.
 - Inherited members include:
 - fields (instance variables)
 - methods
- ☾ However, some elements are not inherited:
 - constructors
 - private members
 - Private members exist within the superclass but cannot be directly accessed by the subclass.

Inheritance - UML



Relationship Between Superclass and Subclass

☾ A subclass interacts with the superclass in several ways:

- Use inherited methods
- Add new methods
- Override inherited methods

A subclass may redefine a method to provide specialized behavior.

Method Overriding

- ☾ Method overriding occurs when a subclass provides its own implementation of a method defined in the superclass.
- ☾ Notation `@Override` indicates that the method overrides a superclass method.

```
@Override  
public void withdraw(double amount) {  
    if (amount <= balance)  
        balance -= amount;  
}
```

Constructors in Subclasses

- Constructors behave differently from ordinary methods in an inheritance hierarchy. A subclass does not inherit the constructors of its superclass, but it must still ensure that the superclass portion of the object is properly initialized.
- When an object of a subclass is created, Java follows a constructor execution chain:
 - The constructor of the superclass is executed first.
 - After the superclass constructor finishes, the constructor of the subclass executes.

This order ensures that all inherited attributes are initialized before the subclass performs its own initialization.

Sample code

```
public class BankAccount {  
  
    protected double balance;  
  
    public BankAccount(double balance){  
        this.balance = balance;  
    }  
}  
  
public class SavingsAccount extends BankAccount{  
  
    public SavingsAccount(double balance){  
        super(balance);  
    }  
}
```